

Apuntes del Curso: Computación Paralela y Distribuida

Prof. José Fiestas

25 de agosto de 2026

1. Modelos computacionales en paralelo (PRAM)

Un **modelo computacional** en paralelo es una abstracción de la funcionalidad de un sistema de acceso, manejo y almacenamiento de información, que es independiente del hardware/software específico que se utiliza.

Define cómo se organizan los cálculos y la comunicación cuando **múltiples procesadores trabajan simultáneamente** para resolver un problema.

En PRAM podemos diseñar inicialmente un algoritmo suponiendo un número muy grande (infinito, ilimitado) de *procesadores*. La transición conceptual puede resumirse como:

RAM $\longrightarrow$ PRAM $\longrightarrow$ Pseudocódigo paralelo $\longrightarrow$ Work y Span $\longrightarrow$ Speedup

Modelo RAM (Random Access Machine)

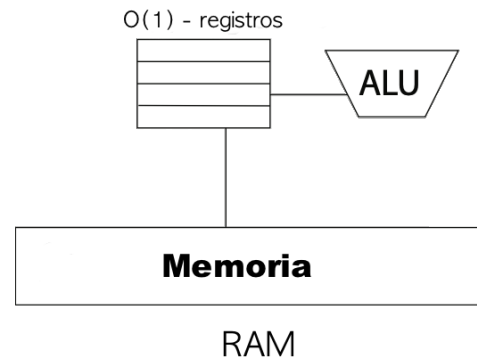
El modelo **RAM** (modelo Von Neumann) es una abstracción de un computador secuencial con memoria infinita, en el cual cada espacio de memoria puede ser accedida en forma directa. Contiene instrucciones para lectura, escritura, y operaciones aritméticas/lógicas. Es útil para el análisis de algoritmos secuenciales.

Por ejemplo, para sumar n números:

```
suma = 0
for i = 0 ... n-1
    suma = suma + A[i]
```

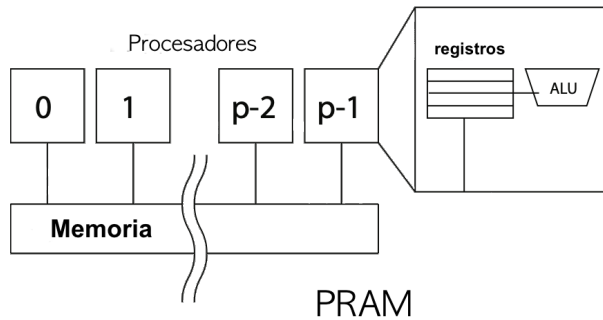
El tiempo de ejecución es:

$$T_s(n) = \Theta(n).$$



Modelo PRAM (Parallel Random Access Machine)

El modelo **PRAM** consiste en un conjunto de *procesadores* idénticos $\{P_1, P_2, \dots, P_n\}$, con acceso a una memoria compartida para lectura/escritura de datos. Cada *procesador* es un RAM, que ejecuta el mismo código en forma sincrónica. No existe una red entre *procesadores*, si no que se comunican a través de la memoria común. Se ignoran las cachés, la latencia real de memoria y los costos físicos de comunicación



Cada *procesador* ejecuta:

- lectura de memoria compartida
- ejecución local
- escritura en memoria compartida

Mientras que conceptualmente:

$$\text{RAM} = 1 \text{ procesador} + \text{memoria}$$

en PRAM tenemos:

$$\text{PRAM} = p \text{ procesadores} + \text{memoria compartida}$$

Clasificación PRAM

- **EREW:** exclusive read exclusive write. Cada locación de memoria puede ser leída o escrita por un solo *procesador* a la vez. Por ejemplo:

```
P0 lee A[0]
P1 lee A[1]
P2 lee A[2]
```

es permitido, mientras que:

```
P0 lee A[5]
P1 lee A[5]
```

no es permitido.

EREW es uno de los modelos PRAM más restrictivos. (E.g. suma de vectores, reducción binaria de suma)

- **CREW:** concurrent read exclusive write. Múltiples *procesadores* pueden leer la misma dirección de memoria pero solo uno puede escribir en ella durante el mismo paso. Por ejemplo, para calcular $B_i = A_i + x$, todos los *procesadores* leen simultáneamente la misma posición que contiene x

$$P_0, P_1, \dots, P_{n-1} \rightarrow x$$

, por lo que existe lectura concurrente.

Sin embargo, cada *procesador* escribe en una posición distinta $P_i \rightarrow B_i$

Otros ejemplos: búsqueda en paralelo de un valor en un arreglo, broadcast

- **ERCW:** exclusive read concurrent write . Se permite escritura simultánea, pero la lectura es exclusiva en el mismo paso. No es muy usada
- **CRCW:** concurrent read concurrent write. Múltiples *procesadores* pueden leer y escribir en el mismo paso (E.g. Global OR) Por ejemplo:

P0: A[5] = 7

P1: A[5] = 12

P2: A[5] = 4

Existe una jerarquía según:

$$EREW \subset CREW \subset CRCW$$

Escritura simultánea **CW**, exige resolver el problema de escritura en el mismo espacio de memoria:

- **modelo común:** todos los *procesadores* escriben la misma data
- **modelo arbitrario:** todos los *procesadores* escriben un dato arbitrario en el mismo paso
- **modelo combinado:** ejecuta la acumulación de los resultados escritos arbitrariamente por cada *procesador* en el mismo espacio de memoria y en el mismo paso.
- **modelo prioritario:** los *procesadores* tienen una prioridad de acceso, y aquel con la mayor prioridad podrá escribir en memoria.

Métricas PRAM

Tiempo de ejecución en paralelo: designa el número de pasos que ejecuta un algoritmo usando p *procesadores*. Cada paso puede ser de lectura, escritura o de computo local.

$$T_p(n)$$

Costo: representa la cantidad total de recursos computacionales consumidos durante la ejecución.

$$C_p(n) = T_p(n)P(n)$$

El **Span** (T_∞), corresponde al tiempo del algoritmo suponiendo un número ilimitado de *procesadores* (longitud del camino crítico)

$$T_\infty = \text{longitud del camino crítico}$$

Trabajo: corresponde a la cantidad total de operaciones realizada por el algoritmo.

$$W(n)$$

Número de *procesadores*:

$$P(n)$$

Un algoritmo es de **costo óptimo** si

$$C_p(n) = T_p(n)P(n) = \Theta(T_s(n))$$

2. Paradigma Work-Time

El principio work-time en paralelismo señala que el diseño de un algoritmo paralelo se analiza bajo dos criterios:

- Trabajo total mínimo (W), con W: cantidad total de operaciones que hace el algoritmo (algoritmo secuencial óptimo)
- Tiempo paralelo o profundidad mínima, T: número de pasos paralelos necesarios, asumiendo una cantidad ilimitada/infinita de *procesadores*.

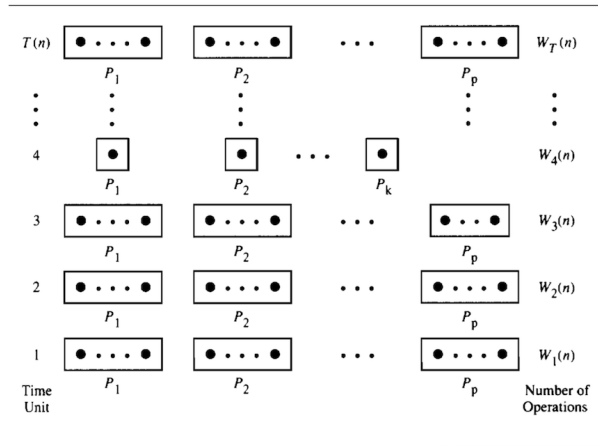


Figura 1: Paradigma WT (work-time scheduling principle [1] Fig.1.9 p.29)

Si conocemos el paralelismo del algoritmo ideal, ¿cómo estimamos su tiempo $T_p(n, p)$ cuando solamente disponemos de p procesadores?

$$p \ll n$$

Para el cálculo de $T_p(n, p)$:

Límite inferior: cantidad de operaciones concurrentes por paso $\geq \frac{W(n)}{p}$

Límite superior: dados p procesadores, y un algoritmo con span $T_\infty(n)$, y trabajo $W(n)$ (cantidad total de operaciones), un algoritmo PRAM puede ejecutarse en

$$T_p(n) \leq \left\lceil \frac{W(n)}{p} \right\rceil + T_\infty(n)$$

Teorema de Brent:

$$T(n, p) = \sum_i \left\lceil \frac{W_i(n)}{p} \right\rceil \leq \left\lfloor \frac{W(n)}{p} \right\rfloor + T_\infty(n)$$

Demostración segun [1]:

Sea $W_i(n)$ la cantidad de operaciones ejecutadas en el paso i , donde $1 \leq i \leq T_\infty(n)$. Si se ejecuta cada $W_i(n)$ operaciones en $\leq \lceil \frac{W_i(n)}{p} \rceil$ pasos en paralelo por los p procesadores. Entonces, el algoritmo PRAM logrará ejecutarse en $T(n, p) = \sum_i \lceil \frac{W_i(n)}{p} \rceil \leq \sum_i (\lfloor \frac{W_i(n)}{p} \rfloor + 1) = \lfloor \frac{W(n)}{p} \rfloor + T_\infty(n)$ pasos en paralelo.

Dado un algoritmo paralelo con span $T_\infty(n)$ y un total de $W(n)$ operaciones (tareas), este puede simularse en un PRAM de p procesadores en

$$T_p(n, p) = O \left(\underbrace{\frac{W(n)}{p}}_{\text{trabajo por procesador}} + \underbrace{T_\infty(n)}_{\text{dependencias}} \right)$$

Esto muestra que el tiempo paralelo está limitado por dos factores:

- la cantidad total de trabajo ($W(n)$)
- la longitud del camino crítico que no puede paralelizarse $T_\infty(n)$

Note que ambos dependen solo de n

Otra idea de demostración

Una forma más precisa de obtener la cota consiste en considerar que durante los T_∞ pasos asociados al camino crítico se realiza al menos una unidad de trabajo.

Quedan entonces:

$$W - T_\infty$$

unidades adicionales de trabajo. Distribuyéndolas entre p procesadores:

$$T_p \leq T_\infty + \frac{W - T_\infty}{p}.$$

Por tanto:

$$T_p \leq \frac{W - T_\infty}{p} + T_\infty$$

Equivalentemente:

$$T_p \leq \frac{W}{p} + \left(1 - \frac{1}{p}\right) T_\infty.$$

Asintóticamente:

$$T_p = O\left(\frac{W}{p} + T_\infty\right)$$

El **costo** (máximo) está definido como

$$C_p(n, p) = T_p(n, p) \cdot p = O(W(n) + T_\infty(n) \cdot p)$$

$W(n)$ y $C_p(n)$ coinciden cuando $p = O\left(\frac{W(n)}{T_\infty(n)}\right)$

W/p representa cómo se reparte idealmente el trabajo entre los *procesadores*, mientras T_∞ representa el límite impuesto por las dependencias entre operaciones.

Es decir, aunque se disponga de muchísimos *procesadores*, no se puede bajar del tiempo T_∞ , porque hay partes del algoritmo que dependen unas de otras. Por lo que se busca equilibrio entre el trabajo en paralelo y la ruta crítica (Span)

Por ejemplo:

$W=1000$ operaciones, $T=20$ pasos paralelos. Con $p = 10$ *procesadores*:

$$T_p \approx 1000/10 + 20 = 120$$

Con $p = 100$ *procesadores*:

$$T_p \approx 1000/100 + 20 = 30$$

Hay que recordar que Brent es un modelo teórico donde **no existe comunicación**, y la **sincronización es implícita**.

Modelos formales PRAM

Pseudocódigo secuencial:

Input: an RE r and a string $T = a_1 \dots a_n$, where $a_i \in \Sigma$.
Output: Match(r, T).

- 1: Generate a T-NFA $M = (Q, \Sigma, \delta, q_0, q_f)$ from r .
- 2: Sort Q in topological order.
- 3: **for all** $q \in Q$ **do**
- 4: Start[q] $\leftarrow 0$
- 5: **end for**
- 6: Compute $Q_f = \{q \mid q \text{ is an } s\text{-state such that there is an } \varepsilon\text{-path from } q \text{ to } q_f\}$
- 7: **for** $i = 1$ to n **do**
- 8: UpdateStart(M , Start, i)
- 9: $pos = \max_{q \in Q_f} \text{Start}[q]$
- 10: **if** $pos > 0$ **then**
- 11: Output (pos, i)
- 12: **for all** $q \in Q$ **do**
- 13: **if** Start[q] $\leq pos$ **then**
- 14: Start[q] $\leftarrow 0$
- 15: **end if**
- 16: **end for**
- 17: **end if**
- 18: **end for**

Figura 2: *

Ejemplo de pseudocódigo. De: Yamamoto, 2018, *A faster algorithm for finding shortest substring matches of a regular expression*

Pseudocódigo paralelo:

El lenguaje formal utiliza **pardo** (do in parallel).

Condición: **n=p**

Algorithm 1: Se asignan n elementos de A a B en paralelo

- 1 **for** $1 \leq i \leq n$ **pardo**
- 2 $B(i) \leftarrow A(i);$

Es decir, si $n=p$ (un elemento por *procesador*).

Significa que las n iteraciones son independientes y pueden ejecutarse concurrentemente, i.e. el bloque **for** se ejecuta en forma **concurrente**.

I.e. *procesador* P1 asigna $B(1)$ a $A(1)$, *procesador* P2 asigna $B(2)$ a $A(2)$, etc

Existen variaciones en la notación:

Algorithm 1 Parallel Regular Expression Matching (PaREM)

```

%Input: Transition table Tt, set of final states F, input T%
%Output: Result of REM%
1:  $I = \text{vector}(p)$  /* initialize final result vector */
   % $P_0 \dots P_p \dots$  processing unit,  $p$  is the total number of
   processing units %
2: for  $P_0, P_1, \dots, P_p$  do in parallel
3:    $start\_position = i * (T.length/p)$ 
4:    $pi\_input = \text{substring}(start\_position, T.length/p)$ 
   %start find possible initial states %
5:   for  $q_0, q_1, \dots, q_n$  do
   %  $pi\_input.at(0)$  returns the first char of  $pi\_input$  %
6:     if  $(Tt[q_i][pi\_input.at(0)] \in Q)$  then
7:        $S[i] = q_i$ 
8:     end if
9:   end for
10:  for  $q_0, q_1, \dots, q_n$  do
   %  $pi\_input.back()$  returns the last char of  $pi\_input$  %
11:    if  $(Tt[q_i][pi\_input.back()] \in Q)$  then
12:       $L[i] = Tt[q_i][pi\_input.back()]$ 
13:    end if
14:  end for

```

Figura 3: *

Ejemplo de pseudocódigo en paralelo. De: *Memeti & Pllana, 2015, PaREM: A Novel Approach for Parallel Regular Expression Matching*

PRAM Caso 1: Global OR

El problema *Global OR* consiste en determinar si al menos uno de los n elementos booleanos de un arreglo es verdadero. Sea el arreglo

$$A = [a_0, a_1, \dots, a_{n-1}], \quad a_i \in \{0, 1\}.$$

Se desea calcular

$$R = a_0 \vee a_1 \vee \dots \vee a_{n-1}.$$

Algorithm 2: Parallel Global OR

```

1 Input: array  $x[1, \dots, n]$  ;
2 Output: Result;
3 Result  $\leftarrow 0$  ;
4 for  $i = 1, \dots, n$  parallel do
5   if  $x[i]$  then
6     Result  $\leftarrow 1$  ;

```

Cada *procesador* evalúa la condición **if** y modifica el valor de Result si $x[i] = 1$

Disponiendo de n procesos, todos los elementos del arreglo son examinados simultáneamente

$$\boxed{T_{\infty}(n) = \Theta(1)}.$$

El trabajo total realizado por los n procesos es

$$\boxed{W(n) = \Theta(n)}.$$

speedup es

$$S_n = \frac{T_1(n)}{T_n(n)} = \frac{\Theta(n)}{\Theta(1)} = \boxed{\Theta(n)}.$$

La eficiencia es

$$E_n = \frac{S_n}{n} = \boxed{\Theta(1)}.$$

Por tanto, bajo el modelo ideal Common CRCW, Global OR puede obtener un *speedup* lineal.

PRAM Caso 2: Global AND

Algorithm 3: Parallel Global AND

```

1 Input: array  $x[1, \dots, n]$  ;
2 Output: Result;
3 Result  $\leftarrow 1$  ;
4 for  $i = 1, \dots, n$  pardo
5   | if not  $x[i]$  then
6   |   | Result  $\leftarrow 0$ ;

```

Cada *procesador* evalúa la condición **if** y modifica el valor de Result si $x[i] = 0$. Análisis de complejidad es similar a Global-OR

PRAM Caso 2: Máximo de n números (CRCW)

Algorithm 4: Cálculo del máximo de n números

```

1 Input: array  $A[1, \dots, n]$  ;
2 Output: array  $M[1, \dots, n]$   $M[i] = 1$  iff  $A[i] = \max_j A[j]$  ;
3 forall  $(i, j) \in \{1, \dots, n\}^2$  pardo
4   |  $B[i, j] \leftarrow A[i] \geq A[j]$  ;
5 forall  $i \in \{1, \dots, n\}$  pardo
6   |  $M[i] \leftarrow \bigwedge_{j=1}^n B[i, j]$  ;

```

1. Cada *procesador* compara el valor de un elemento del array con otro del mismo array A
2. El índice del elemento de A donde el Global AND del array booleano M sea 1 será el máximo de A

PRAM Caso 4: Suma de n números

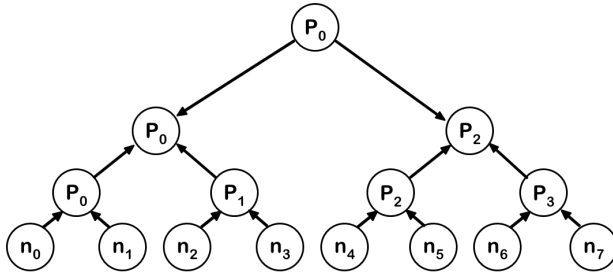
Algorithm 5: Suma de n números en paralelo

```

1 Input: vector  $A$  con  $n = 2^k$  elementos ;
2 Output: suma  $S$  de elementos de  $A$  ;
3 for  $i = 1$  to  $n$  pardo
4    $B[i] \leftarrow A[i]$  ;                                     // paso 1
5 for  $h = 1$  to  $\log(n)$  do
6   for  $i = 1$  to  $n/2^h$  pardo
7      $B[i] \leftarrow B[2i - 1] + B[2i]$  ;                   // paso 2
8  $S \leftarrow B[1]$  ;                                       // paso 3

```

Para $n=8$, $p=8$



En el paso 1 se ejecutan n operaciones

En el paso 2 se ejecutan $\sum_{j=1}^{\log(n)} n/2^j$

En el paso 3 se ejecuta 1 operación

Por lo tanto:

Trabajo:

$$W(n) = W_1(n) + W_2(n) + W_3(n) = n + \sum_{j=1}^{\log(n)} n/2^j + 1 = O(n)$$

Tiempo (span):

$$T_{\infty}(n) = O(\log(n) + 2) = O(\log(n))$$

Costo:

$$C_p(n) = O(n + p \log n)$$

Por lo tanto, el algoritmo es WT-óptimo

$$T_p(n, p) = O\left(\frac{n}{p} + \log n\right)$$

Este algoritmo ejecuta un modelo PRAM CREW

PRAM: Suma de n números para un procesador

Algorithm 6: Suma de n números para un procesador

```
1 Input:  $A[1, \dots, n]$ ,  $p$  procesadores,  $s$  el procesador ID ( $1 \leq s \leq p$ );  
2 Output: suma de elementos de A almacenada en  $S = \sum_{i=1}^n A[i]$  ;  
3 for  $j = 1$  to  $l = n/p$  do  
4    $B(l(s-1) + j) \leftarrow A(l(s-1) + j);$   
5 for  $h = 1$  to  $\log(n)$  do  
6   if  $k - h - q \geq 0$  then  
7     for  $j = 2^{k-h-q}(s-1) + 1$  to  $2^{k-h-q}s$  do  
8        $B(j) \leftarrow B(2j-1) + B(2j);$   
9   else  
10    if  $s \leq 2^{k-h}$  then  
11       $B(s) \leftarrow B(2s-1) + B(2s);$   
12 if  $s = 1$  then  
13    $S \leftarrow B(1);$ 
```

Extensiones del modelo PRAM

Recordando. El modelo PRAM constituye una abstracción fundamental para el diseño y análisis de algoritmos paralelos. Los modelos clásicos (EREW, CREW, CRCW) solo distinguen lectura/escritura concurrente, y consideran:

- p procesadores (infinitos)
- una memoria global compartida
- procesadores sincronizados
- operaciones elementales y acceso a memoria de costo constante
- comunicación implícita mediante memoria compartida

PRAM supone idealmente:

$$T_{\text{memoria}} = O(1)$$

y no representa explícitamente:

$$T_{\text{comunicacion}}, \quad T_{\text{sincronizacion}}, \quad T_{\text{latencia}}.$$

Para reducir estas idealizaciones aparecen diferentes extensiones de PRAM, como:

APRAM,	Phase PRAM,	LPRAM,	BPRAM
--------	-------------	--------	-------

Cada una introduce un aspecto diferente de las arquitecturas paralelas reales.

APRAM: Asynchronous PRAM

APRAM (*Asynchronous Parallel Random Access Machine*) modifica uno de los supuestos fundamentales de PRAM: la sincronización global de los procesadores. En PRAM clásico todos los procesadores avanzan a pasos sincronizados. En APRAM los procesadores pueden avanzar a velocidades diferentes.

tiempo --->

P0: |---op1---|---op2---|---op3---|

P1: |--op1--|-----op2-----|--op3--|

P2: |-----op1-----|--op2--|----op3----|

Los procesadores ejecutan instrucciones independientemente y solamente necesitan sincronizarse cuando existen dependencias entre operaciones.

Por tanto, APRAM considera algoritmos donde los procesadores progresan independientemente, las operaciones tienen duraciones diferentes. La sincronización debe hacerse explícita, por lo que pueden aparecer esperas debido a dependencias.

Ejemplo:

Dos procesos suman los elementos de un array. P_0 suma los primeros $n/2$ elementos en s_0 , mientras

que P_1 suma los restantes $n/2$ elementos en $s1$. Por ello, la suma total S debe sincronizar las sumas parciales antes de ejecutar $S = s0 + s1$.

En pseudocódigo APRAM:

```
P0:
    s0 = A[0] + A[1] + ... + A[n/2-1]
    wait(flag1 == true)
    S = s0 + s1

P1:
    s1 = A[n/2] + ... + A[n-1]
    flag1 = true
```

Phase PRAM

Phase PRAM introduce una solución intermedia entre PRAM (completamente síncrono) y APRAM (completamente asíncrono). La ejecución se divide en **fases**.

Durante cada fase los procesadores realizan trabajo paralelo. Al terminar la fase se ejecuta una sincronización global. Por tanto

cómputo paralelo $\rightarrow$ barrera $\rightarrow$ cómputo paralelo $\rightarrow$ barrera

Los procesadores pueden trabajar de manera independiente dentro de una fase, pero ningún procesador comienza la siguiente fase hasta que todos hayan terminado la anterior.

Ejemplo: reducción binaria:

Considere una reducción:

```
while m > 1
    for i = 0 ... m/2-1 pardo
        B[i] = A[2*i] + A[2*i+1]
    BARRIER
    A = B
    m = m/2
```

Cada nivel del árbol constituye una fase.

Para n elementos existen:

$$\log n$$

niveles y, por tanto:

$$\Theta(\log n)$$

fases de sincronización.

Phase PRAM permite considerar el costo de sincronización. Si una barrera cuesta:

$$T_{\text{barrier}},$$

y existen S fases, entonces solamente el costo de las barreras puede ser aproximadamente:

$$T_{\text{sync}} = ST_{\text{barrier}}.$$

Por tanto:

$$T_p = T_{\text{comp}} + T_{\text{sync}}$$

LPRAM: Local-memory PRAM

Dado que en un computador real existen diferentes niveles de memoria, y cada procesador dispone de una memoria local, en LPRAM se reconoce que:

$$T_{\text{local}} < T_{\text{global}}$$

Por tanto, el costo de un algoritmo depende no solamente del número de operaciones, sino también de la ubicación de los datos.

Un algoritmo que reutiliza datos locales puede ser considerablemente más eficiente que uno que realiza constantemente accesos no locales.

Esta idea aparece posteriormente en:

- jerarquías de caché;
- arquitecturas NUMA;
- memoria local de GPUs;
- sistemas distribuidos;
- particionamiento de datos.

BPRAM: Block PRAM

BPRAM introduce la transferencia de datos en bloques. E.g la transferencia de un bloque:

$$A[i], A[i + 1], \dots, A[i + b - 1].$$

de tamaño b

El costo de transferir un bloque puede modelarse conceptualmente como:

$$T_{\text{transferencia}} = \alpha + \beta b$$

donde:

α : costo fijo de iniciar la transferencia o latencia (orden de microsegundos)

β : costo por elemento transferido (orden de nanosegundos)

b : cantidad de datos transferidos.

Esta expresión permite distinguir la **latencia** del costo asociado al volumen de datos **ancho de banda**

Ejemplo:

Para transferir n datos, si realizamos n transferencias independientes de un elemento:

$$T_1 = n(\alpha + \beta).$$

Si transferimos todos los datos mediante un bloque:

$$T_2 = \alpha + \beta n.$$

Generalmente:

$$\alpha \gg \beta,$$

Modelos derivados

agrupar los datos puede reducir considerablemente el costo.

Los cuatro modelos anteriores modifican diferentes supuestos del modelo PRAM.

Modelo	Supuesto	Extensión
APRAM	Sincronización global	Asincronía
Phase PRAM	Sincronización gratuita	Fases y barreras
LPRAM	Acceso uniforme a memoria	Localidad
BPRAM	Acceso palabra por palabra	Transferencia por bloques

Los modelos anteriores permiten introducir progresivamente cuatro problemas fundamentales:

asincronía, sincronización, localidad, comunicación.

Sin embargo, para analizar sistemas paralelos reales necesitamos modelos que combinen varios de estos factores.

Dos modelos particularmente importantes son:

BSP y LogP.

BSP: Bulk Synchronous Parallel

BSP (*Bulk Synchronous Parallel*) organiza la ejecución de un algoritmo paralelo mediante una secuencia de fases denominadas:

supersteps.

Cada procesador dispone de capacidad de cómputo, memoria local, capacidad de comunicación. Un superstep contiene conceptualmente:

cómputo local → comunicación → barrera

```

P0: [ computo ] [ comunicacion ] |
P1: [ computo ] [ comunicacion ] |
P2: [ computo ] [ comunicacion ] | BARRIER
P3: [ computo ] [ comunicacion ] |
    |
    siguiente
    superstep

```

Al final del superstep existe una barrera global.

Una máquina BSP puede caracterizarse mediante:

$$(p, g, L)$$

donde:

p : número de procesadores.

g : parámetro asociado al costo de comunicación por palabra.

L : costo asociado a la sincronización global entre supersteps.

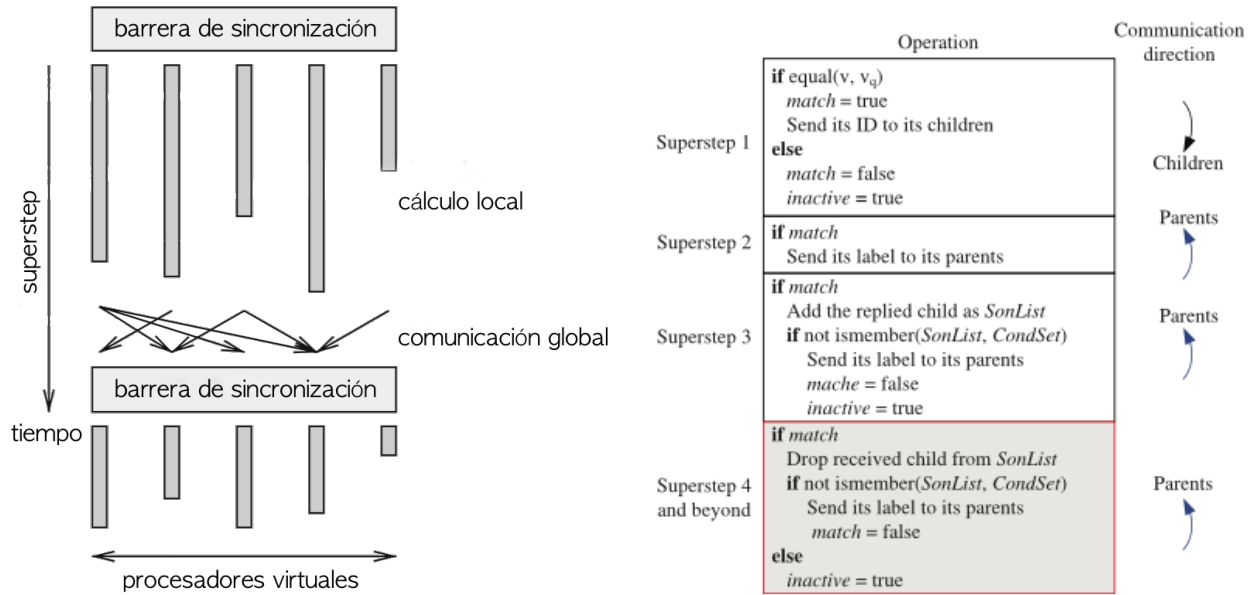


Figura 4: Algoritmo BSP para un problema de grafos. Fuente: PRS: J.Wang, 2015, Parallel Relaxation Simulation for Massive Graphs

El **costo de un superstep** puede modelarse como:

$$T_{\text{superstep}} = w + gh + L$$

donde w es el máximo trabajo local realizado por algún procesador durante el superstep, h es el máximo número de palabras enviadas o recibidas por algún procesador. Por tanto:

$$T_{\text{superstep}} = \underbrace{w}_{\text{cómputo}} + \underbrace{gh}_{\text{comunicación}} + \underbrace{L}_{\text{sincronización}}$$

Si existen S supersteps:

$$T_{\text{BSP}} = \sum_{s=1}^S (w_s + gh_s + L).$$

Por tanto:

$$T_{\text{BSP}} = \sum_{s=1}^S w_s + g \sum_{s=1}^S h_s + SL$$

BSP recoge varias ideas introducidas anteriormente:

Phase PRAM	→	supersteps y barreras
LPRAM	→	cómputo y memoria local
BPRAM	→	volumen de comunicación

Por tanto, conceptualmente:

$$\text{BSP} \approx \text{Phase PRAM} + \text{localidad} + \text{comunicación} + \text{sincronización}$$

Ejemplo: reducción BSP

Supongamos que cada uno de los p procesadores posee inicialmente un valor:

$$x_0, x_1, \dots, x_{p-1}.$$

Deseamos calcular:

$$S = \sum_{i=0}^{p-1} x_i.$$

Algorithm 7: Reducción paralela de suma

Input: Arreglo $A[0, \dots, n-1]$
Output: $A[0] = \sum_{i=0}^{n-1} A[i]$

```

1  $step \leftarrow 1$ 
2 while  $step < n$  do
3   for  $i = 0$  to  $\lfloor n/(2 \cdot step) \rfloor - 1$  pardo
4   |  $A[2 \cdot step \cdot i] \leftarrow A[2 \cdot step \cdot i] + A[2 \cdot step \cdot i + step]$ 
5   | BSP_SYNC
6   |  $step \leftarrow 2 \cdot step$ 
```

Mediante una reducción binaria se requieren:

$$\log_2 p$$

niveles.

Cada nivel puede modelarse como un superstep BSP.

Si cada procesador activo realiza una suma y comunica una palabra: $w = h = O(1)$

Complejidad:

Superpaso : $\log p$ superpasos

En cada uno:

cada procesador envía 1 dato

cada procesador recibe 1 dato

Entonces:

w=1, h=1

tiempo por superpaso: $T_{sp} = 1 + g + L$

Tiempo total de reducción: $T_{red} = \log p(1 + g + L)$

$$T_{BSP} = O\left(\frac{n}{p} + g \log p + L \log p\right)$$

En PRAM habríamos obtenido solamente:

$$T_p = O(\log p).$$

BSP muestra que cada nivel implica además comunicación y sincronización.

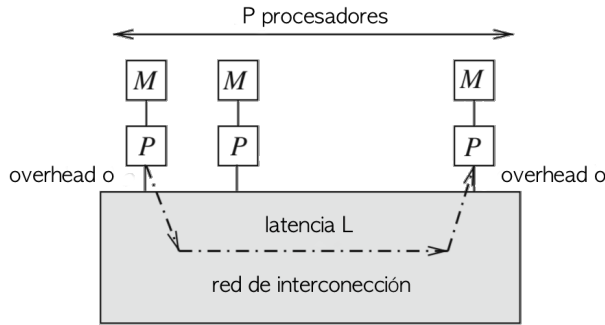
LogP

Ya que muchos sistemas paralelos no necesitan realizar sincronizaciones globales después de cada fase, i.e. los procesadores pueden trabajar independientemente, enviar mensajes, continuar realizando cómputo, recibir información posteriormente, su comportamiento se aproxima más a APRAM.

LogP introduce un modelo para representar este tipo de ejecución asíncrona considerando explícitamente los costos de comunicación. El nombre LogP proviene de cuatro parámetros:

$$L, o, g, P$$

donde:



L (**latency**) tiempo máximo asociado al tránsito de un mensaje pequeño desde el procesador origen hasta el procesador destino.

o (**overhead**) tiempo durante el cual el procesador está ocupado gestionando el envío o recepción de un mensaje.

g (**gap**) intervalo mínimo entre transmisiones o recepciones consecutivas. Representa una limitación asociada al ancho de banda.

P : número de procesadores.

Algorithm 8: Reducción paralela de suma para el modelo LogP

Input: Arreglo $A[0, \dots, n-1]$

Output: $A[0] = \sum_{i=0}^{n-1} A[i]$

```
1  $d \leftarrow 1$ 
2 while  $d < n$  do
3   for  $i = 0$  to  $\lfloor n/(2d) \rfloor - 1$  pardo
4      $x \leftarrow A[2di + d]$ 
5     send( $x, 2di$ )
6      $y \leftarrow \text{receive}()$ 
7      $A[2di] \leftarrow A[2di] + y$ 
8    $d \leftarrow 2d$ 
```

Complejidad

Reducción log p

While indica log p iteraciones. En cada iteración cada *procesador* activo envía y recibe 1 mensaje. Además, realiza 1 suma.

El costo de una iteración es aproximadamente $T = L + o + g$

Total reducción:

$$T_{red} = \log p(L + o + g)$$

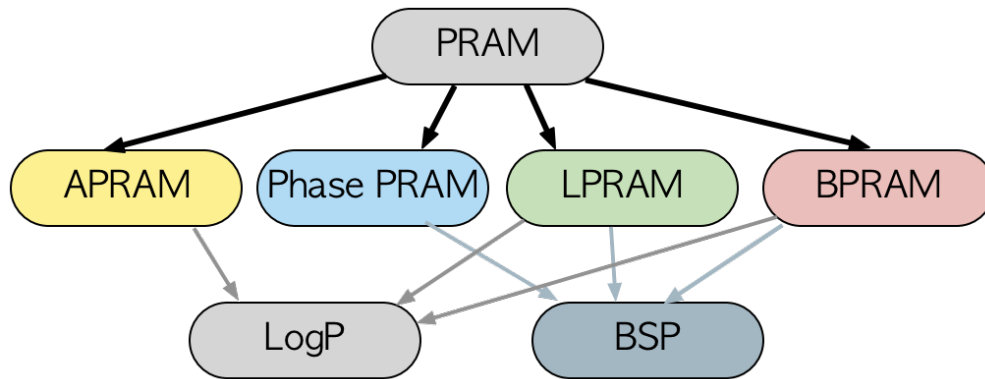
$$T_{\log P} = O\left(\frac{n}{p} + \log p(L + o + g)\right)$$

donde $\frac{n}{p}$ es el cómputo local, $o \log p$ es el overhead CPU, $L \log p$ es la latencia, y $g \log p$ es el ancho de banda

LogP separa el costo de comunicación con mayor detalle.

Relación global entre los modelos

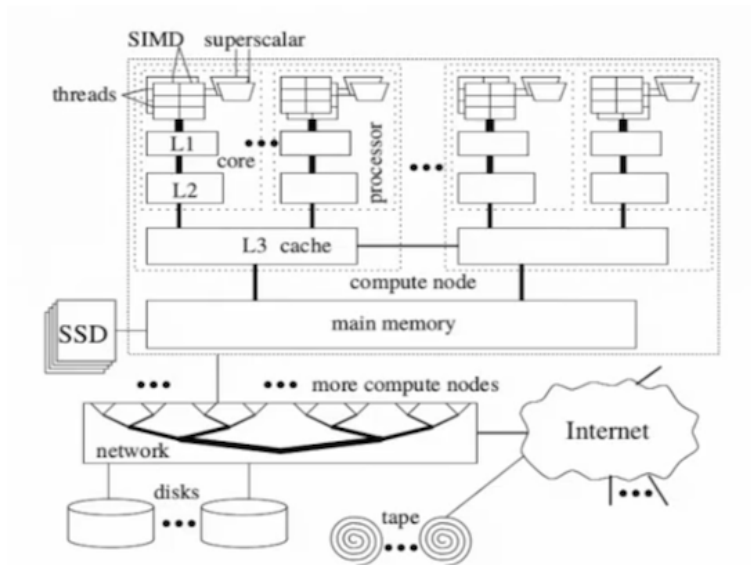
Los modelos estudiados no forman necesariamente una única secuencia lineal. Cada extensión de PRAM introduce una preocupación diferente



APRAM	: asincronía
Phase PRAM	: sincronización
LPRAM	: localidad
BPRAM	: volumen y granularidad de comunicación
BSP	: cómputo + comunicación + sincronización
LogP	: asincronía + latencia + overhead + ancho de banda

Niveles de Paralelismo

Estrategia es crear modelos con pocos niveles de paralelismo (híbridos)



Relación DAG-PRAM

DAG → **PRAM**:

Cada nivel del DAG que puede ser paralelizado, se representa como un modelo PRAM

PRAM → **DAG**:

Identificar bucles y condicionales con distintos *procesadores* en PRAM

DAG → Red:

- Nodos son *procesadores*
- Aristas son líneas de comunicación en la red
- Tiempo de ejecución depende de los nodos, aristas, y de la longitud de las rutas en cada nivel del DAG.

Referencias

[1] Joseph Jája *An introduction of parallel algorithms*. University of Maryland Ch1.3.2., 1.5.